

DBMS -- Types of Database Systems

Target: Explain all 4 DBMS types in 40s. Clarify why ER Model is NOT a DBMS type.

First -- A Quick Correction

You mentioned reading "Hierarchical, Network, Relational, ER Model" as the 4 types.

ER Model is NOT a type of DBMS.

It is a data modeling / design tool -- a way to draw and plan your database before building it.

The 4th actual type of DBMS is Object-Oriented DBMS (OODBMS).

Correct list:

1. Hierarchical DBMS
2. Network DBMS
3. Relational DBMS (RDBMS) most used today
4. Object-Oriented DBMS (OODBMS)

We'll cover ER Model separately at the end so you can explain both correctly.

The 4 Types -- Side-by-Side

Type
Structure
Navigation
Real Examples

Hierarchical
Tree (parent children)
Top-down only
IBM IMS, Windows Registry

Network
Graph (records + sets, many-to-many)
Any direction via pointers
IDS, IDMS

Relational
Tables (rows + columns)
SQL queries
MySQL, PostgreSQL, Oracle

Object-Oriented
Objects with attributes + methods
OQL / object traversal
db4o, ObjectDB

TYPE 1 -- Hierarchical DBMS

What it is

Data is organized as a tree. One parent can have many children, but each child has exactly one parent.

Company
 / \
 Dept-A Dept-B
 / \
 Emp1 Emp2 Emp3
Key Characteristics

Relationship: One-to-Many (1:N) only

Navigation: Must start from the root and go down

Data is stored as physical pointers between records

Very fast for predictable, top-down queries (e.g., "get all employees in Dept-A")

The Big Problem

No Many-to-Many support -- an employee can't belong to two departments

Rigid structure -- changing the hierarchy requires restructuring all data
Redundancy: if two parents share a child, the child must be duplicated

Real-World Analogy

Think of a company org chart -- you start at the CEO and drill down. You can't jump sideways or skip levels.

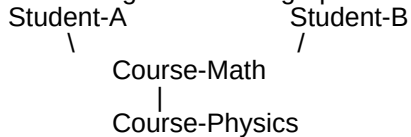
Where it's still used

IBM IMS (mainframe banking systems)
Windows Registry (keys have parents, values are children)
DNS (domain name hierarchy: .com google.com mail.google.com)

TYPE 2 -- Network DBMS

What it is

Data is organized as a graph. Records are called nodes, relationships are called sets. A child can have multiple parents.



Key Characteristics

Relationship: Many-to-Many (M:N) supported -- unlike hierarchical
Navigation: Via pointers in any direction
Defined by the CODASYL standard (1969)
More flexible than hierarchical

The Big Problem

Complex navigation -- programmer must know the exact physical path to the data
No declarative query language (no SQL equivalent) -- you write navigational code
Schema changes are very hard -- pointers must be manually updated

Real-World Analogy

Think of a road map -- you can go from any city to any other city via multiple routes. But you have to know and code the exact routes yourself.

Where it's still used

Mostly legacy mainframe systems (IDMS)
Conceptually similar to modern graph databases (Neo4j) -- though those are a separate modern category

TYPE 3 -- Relational DBMS (RDBMS)

What it is

Data is organized into tables (relations). Tables are linked via keys, not physical pointers. You query using SQL -- a declarative language.

employees table departments table

emp_id	name	dept_id	dept_id	dept_name
1	Avanish	10	10	HR
2	Rahul	20	20	Finance

FK PK

...

Key Characteristics

Based on Codd's 12 Rules (E.F. Codd, 1970)
Uses SQL -- declarative (you say what, not how)
Supports all relationship types: 1:1, 1:N, M:N (via junction tables)
ACID transactions: Atomicity, Consistency, Isolation, Durability
Strong normalization support

Why it won

No need to know physical storage paths -- SQL handles it
Flexible: any query without restructuring data
Mature ecosystem, strong tooling, wide adoption

Real-World Examples

MySQL, PostgreSQL, Oracle, MS SQL Server, SQLite

Real-World Analogy

Think of an Excel workbook with multiple sheets -- each sheet is a table, and you can link them by column values. But unlike Excel, SQL lets you ask complex questions across all sheets in one line.

TYPE 4 -- Object-Oriented DBMS (OODBMS)

What it is

Data is stored as objects -- just like in OOP languages (Java, C++). Each object has attributes (data) and methods (behaviour).

Object: Employee

- empID: 101
- name: "Avanish"
- department: Department object object reference, not FK
- getSalary() method
- getReports() method

Key Characteristics

Supports inheritance, encapsulation, polymorphism at DB level

No impedance mismatch -- objects in code map directly to objects in DB

Supports complex data types (audio, video, spatial data) natively

Uses OQL (Object Query Language) instead of SQL

The Tradeoffs

Less mature than RDBMS

Smaller ecosystem, fewer tools

Not ideal for simple tabular reporting

Where it's used

CAD/CAM systems (engineering designs)

Multimedia databases

Scientific research data (genome databases)

Some modern variants: Object-Relational DBMS (PostgreSQL partially supports this)

Real-World Analogy

Think of saving a Java object directly to disk -- the database stores the exact object with its methods, not just flat columns.

Comparison Table -- All 4 Types

Feature

Hierarchical

Network

Relational

Object-Oriented

Data structure

Tree

Graph

Tables

Objects

Relationship

1:N only

M:N via pointers

Any via SQL

Object references

Query language

Navigational

Navigational

SQL (declarative)

OQL

M:N support

No

[OK] Yes

[OK] Yes (junction table)

[OK] Yes

Schema flexibility

Rigid

Rigid

High

High

Redundancy risk

High

Medium

Low (normalization)

Low

Performance

Fast for known paths

Fast with right pointers

Optimized by query planner

Fast for complex objects

Still used

Legacy/mainframe

Legacy/mainframe

[OK] Dominant

Niche/specialized

The ER Model -- Design Tool, Not a DBMS Type

The Entity-Relationship (ER) Model is a conceptual modeling tool used to design a database before you build it.

Think of it this way:

- ER Model = the blueprint (architecture diagram)

- DBMS = the building (actual database software)

You draw an ER diagram then implement it in a Relational DBMS (or any other type).

```

DESIGN PHASE

IMPLEMENTATION PHASE

ER Diagram

Relational Tables

(entities, attributes, relationships, cardinality)

(CREATE TABLE, FK, normalization)

```

ER Model Quick Recap

Component

Symbol

Represents

Entity

Rectangle

Real-world object (Student)

Attribute

Oval

Property of entity (Name, Age)

Relationship

Diamond

Association between entities

Weak Entity
Double rectangle
Entity that needs parent to exist

Cardinality
1:1, 1:N, M:N
How many of each side

If an interviewer asks about ER Model as a "type", you can gently correct:
"ER Model is actually a conceptual design technique, not a DBMS type itself. The 4 main types are Hierarchical, Network, Relational, and Object-Oriented."

Your 40-Second Script -- DBMS Types

"There are 4 main types. Hierarchical organizes data as a tree -- fast but only supports one-to-many relationships and can't handle many-to-many. Network improves on that by using a graph structure -- supports many-to-many but requires navigational coding to query. Relational is the dominant type today -- data in tables, linked by keys, queried using SQL, with full ACID support. Object-Oriented stores data as objects with attributes and methods, useful for complex data types like CAD or multimedia. Relational DBMS is what we use in production -- MySQL, PostgreSQL, Oracle."

Follow-Up Questions (Expect These)

Q: Why did Relational DBMS replace Hierarchical and Network?

SQL is declarative -- you say what you want, not how to find it. No need to know physical data paths. Far more flexible for ad-hoc queries and schema changes.

Q: What is the impedance mismatch problem?

The mismatch between OOP code (objects) and RDBMS storage (tables). You use ORMs (like Hibernate, Mongoose) to bridge this gap. OODBMS eliminates it entirely.

Q: What is a NoSQL database -- is that a 5th type?

NoSQL is a broad modern category: document stores (MongoDB), key-value (Redis), column-family (Cassandra), graph (Neo4j). They sacrifice some ACID guarantees for scale and flexibility.

Q: What is an Object-Relational DBMS (ORDBMS)?

A hybrid -- relational tables with OOP extensions like custom types, inheritance, and methods. PostgreSQL is the best example.